

stm32f103 workspace notes

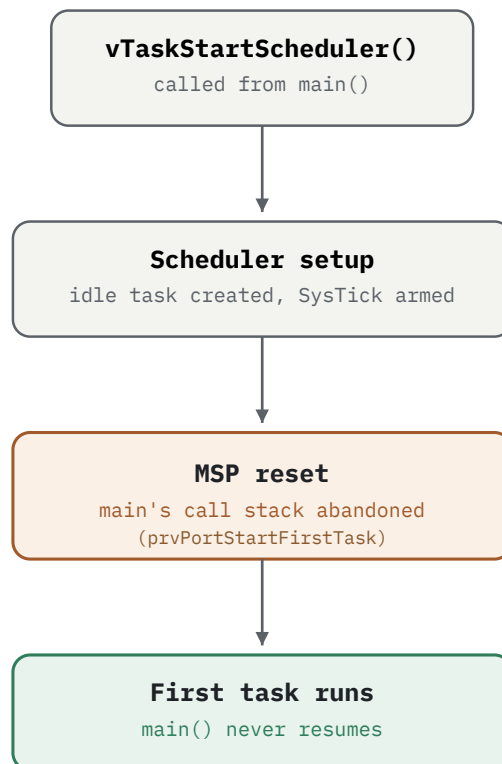
main() and MSP, after vTaskStartScheduler()

Two follow-on questions about the same moment: your own project's `main()` calls `vTaskStartScheduler()` and, from that point forward, nothing about the CPU's stack usage is the same again.

1. What happens to main()?

SHORT ANSWER: IT DOESN'T RETURN - ITS CALL STACK GETS ABANDONED

The `for(;;) {}` loop sitting right after `vTaskStartScheduler()` in your own `main.c` is genuinely unreachable code in the normal case. Here's why.



the only path back to `main()`'s own `for(;;)` loop would require the original MSP value - and step 3 just overwrote it

The `for(;;) {}` after `vTaskStartScheduler()` only ever runs if the function returns before reaching step 3 - in practice, almost always "not enough heap to create the idle task."

- 1 **An ordinary function call.** `main()` calling `vTaskStartScheduler()` is nothing special yet - a normal call, a normal stack frame pushed, same as any other function.
- 2 **Setup runs inside it.** The idle task gets created, SysTick gets armed using `configCPU_CLOCK_HZ`, and eventually `prvPortStartFirstTask()` executes.
- 3 **The pivotal instruction.** `prvPortStartFirstTask()` reads the original reset stack-top value straight out of the vector table and writes it into MSP. Nothing gets popped - MSP is simply relocated back to the top of RAM. Every frame built up along the way (`Reset_Handler`'s, `main()`'s, `vTaskStartScheduler()`'s) is still physically in RAM, but nothing points to it anymore.
- 4 **`svc 0` fires**, landing in `vPortSVCHandler`, which loads the first task's forged initial frame and executes `eret`. Execution lands at that task's entry function, running on *its own* stack, not the one `main()` was ever on.

FreeRTOS/portable/GCC/ARM_CM3/port.c - the actual instruction that abandons `main()`'s stack

`prvPortStartFirstTask:`

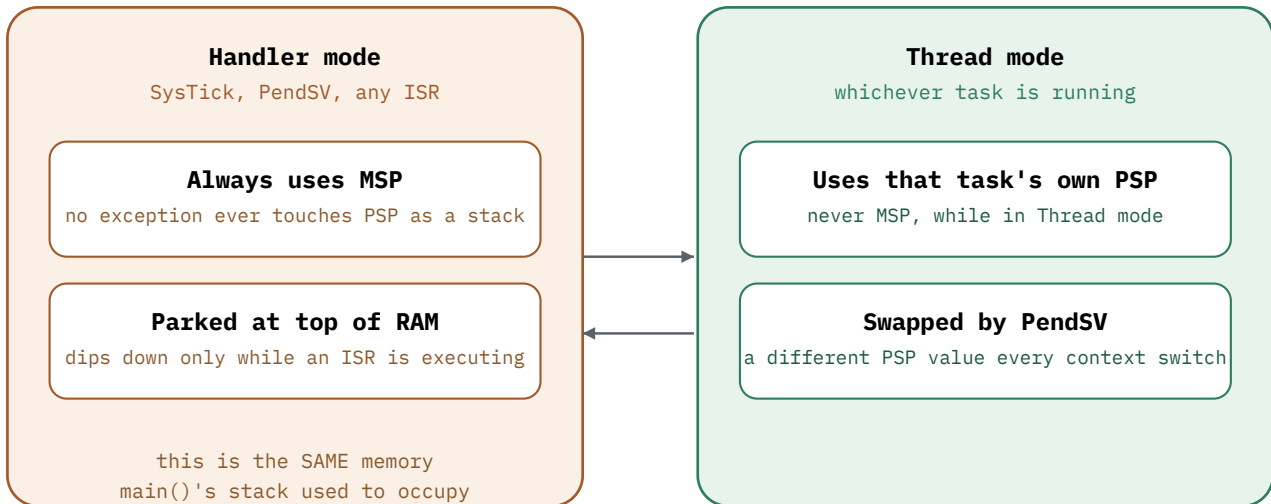
```
ldr r0, =0xE000ED08    ; VTOR - Vector Table Offset Register
ldr r0, [r0]           ; r0 = vector table's base address
ldr r0, [r0]           ; r0 = word[0] = the RESET stack-top value
msr msp, r0            ; MSP relocated back there - nothing popped,
                        ;   main()'s whole call chain just stops mattering

cpsie i
cpsie f
dsb
isb
svc 0                  ; --- jumps into vPortSVCHandler, see below ---
```

2. What happens to MSP afterward?

IT DOESN'T VANISH - IT BECOMES THE EXCEPTION STACK, PERMANENTLY

MSP isn't retired once the scheduler starts - it changes job entirely. From this point on, which stack pointer is "active" depends on what mode the CPU is in, not on which task happens to be running.



Which stack is active is a function of CPU mode (Handler vs. Thread), never of which task happens to be running - Handler mode is hardwired to MSP regardless of what `CONTROL.SPSEL` says.

Two things happen at the exact moment `vPortSVCHandler` executes `eret` :

- **PSP gets loaded** with the first task's stack pointer - explicit, done by the handler's own code.
- **The mode switch happens automatically.** The special `EXC_RETURN` value forced into `r14` (`orr r14, #0xd` in the handler) tells the hardware "return to Thread mode, and use PSP from here on" - the CPU applies this as part of processing the exception return itself, not via a separate instruction.

From then on, MSP sits parked at the top of RAM whenever nothing is interrupting anything - and dips down briefly, in a perfectly ordinary push/pop fashion, every time `SysTick_Handler` or `PendSV_Handler` actually runs. Every ISR in the system - regardless of which task it interrupts - shares this one MSP stack, because Handler mode is *always* MSP-based by the architecture, full stop. `CONTROL.SPSEL` (the bit that picks MSP vs. PSP) only has any effect in Thread mode; Handler mode ignores it entirely.

Worth noticing: `PendSV_Handler`'s own bookkeeping - the couple of registers it pushes with `stmdb sp!, {r3, r14}` around its call into `vTaskSwitchContext()` - happens on *MSP*, because `PendSV_Handler` itself is running in Handler mode. The *PSP* value it reads and writes elsewhere in that same function is being manipulated as *data* (a number it copies in and out of a task's TCB) - not used as the stack the handler's own code is currently executing on. Two completely different stacks, touched by the same function, for two completely different reasons.

stm32f103 workspace notes · continues from the SysTick / PendSV / SVC and task-context notes